

**Московский государственный технический
университет им. Н.Э. Баумана**

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”

Курс «Парадигмы и конструкции языков программирования»

Отчет по лабораторной работе №6

«Шаблоны проектирования и модульное тестирование в Python»

Выполнил:
студент группы ИУ5 - 32Б:
Акопов А.
Подпись и дата:

Проверил:
преподаватель каф. ИУ5
Нардид А. Н.
Подпись и дата:

Москва, 2025 г.

Задание:

1. Необходимо для произвольной предметной области реализовать от одного до трех шаблонов проектирования: один порождающий, один структурный и один поведенческий. В качестве справочника шаблонов можно использовать [следующий каталог](#). Для сдачи лабораторной работы в минимальном варианте достаточно реализовать один паттерн.
2. В модульных тестах необходимо применить следующие технологии:
 - TDD - фреймворк.
 - BDD - фреймворк.
 - Создание Mock-объектов.

Код:

composite.py

```
from abc import ABC, abstractmethod
from typing import List
from enum import Enum

class CarType(Enum):
    SEDAN = "sedan"
    SUV = "suv"
    SPORT = "sport"

class CarComponent(ABC):
    """Абстрактный компонент для паттерна Composite"""

    @abstractmethod
    def get_price(self) -> float:
        pass

    @abstractmethod
    def get_description(self) -> str:
        pass

class CarPart(CarComponent):
    """Автозапчасть (Leaf)"""

    def __init__(self, name: str, price: float):
        self.name = name
        self.price = price

    def get_price(self) -> float:
        return self.price

    def get_description(self) -> str:
```

```
return f"Запчасть: {self.name} - {self.price} руб."
```

```
class Car(CarComponent):  
    """Автомобиль (Leaf)"""
```

```
def __init__(self, model: str, base_price: float, car_type: CarType):  
    self.model = model  
    self.base_price = base_price  
    self.car_type = car_type  
    self.options: List[CarPart] = []
```

```
def add_option(self, option: CarPart):  
    self.options.append(option)
```

```
def get_price(self) -> float:  
    options_price = sum(option.get_price() for option in self.options)  
    return self.base_price + options_price
```

```
def get_description(self) -> str:  
    description = f"Автомобиль: {self.model} ({self.car_type.value}) - {self.base_price} руб."  
    if self.options:  
        description += "\n Доп. опции:"  
        for option in self.options:  
            description += f"\n   - {option.get_description()}"  
    return description
```

```
class CarPackage(CarComponent):  
    """Пакет автомобилей (Composite)"""
```

```
def __init__(self, name: str):  
    self.name = name  
    self.children: List[CarComponent] = []
```

```
def add(self, component: CarComponent):  
    self.children.append(component)
```

```
def remove(self, component: CarComponent):  
    self.children.remove(component)
```

```
def get_price(self) -> float:  
    return sum(child.get_price() for child in self.children)
```

```
def get_description(self) -> str:  
    description = f"Пакет: {self.name}\n"  
    for child in self.children:  
        child_desc = child.get_description()  
        # Добавляем отступы для вложенных элементов  
        for line in child_desc.split("\n"):  
            description += f"   {line}\n"
```

```

        description += f"Общая стоимость пакета: {self.get_price()} руб."
        return description

# Фабрика для создания тестовых данных
def create_sample_autosalon():
    """Создание тестового автосалона"""
    # Создаем запчасти
    leather_seats = CarPart("Кожаные сиденья", 50000)
    navigation = CarPart("Навигационная система", 30000)
    sport_kit = CarPart("Спортивный обвес", 70000)

    # Создаем автомобили
    sedan = Car("Toyota Camry", 2500000, CarType.SEDAN)
    sedan.add_option(leather_seats)
    sedan.add_option(navigation)

    suv = Car("Honda CR-V", 3200000, CarType.SUV)
    suv.add_option(navigation)

    sport_car = Car("Porsche 911", 8500000, CarType.SPORT)
    sport_car.add_option(leather_seats)
    sport_car.add_option(sport_kit)

    # Создаем пакеты
    family_package = CarPackage("Семейный пакет")
    family_package.add(sedan)
    family_package.add(suv)

    premium_package = CarPackage("Премиум пакет")
    premium_package.add(sport_car)

    # Главный пакет автосалона
    autosalon = CarPackage("Автосалон 'Престиж'")
    autosalon.add(family_package)
    autosalon.add(premium_package)

    return autosalon

```

tests/tests_TDD.py

```

import unittest
from composite import CarPart, Car, CarPackage, CarType

class TestAutosalonTDD(unittest.TestCase):
    """TDD тесты для автосалона"""

    def test_car_part_creation(self):
        """Тест создания автозапчасти"""

```

```

part = CarPart("Тормозные колодки", 8000)
self.assertEqual(part.get_price(), 8000)
self.assertIn("Тормозные колодки", part.get_description())

def test_car_creation(self):
    """Тест создания автомобиля"""
    car = Car("Toyota Camry", 2500000, CarType.SEDAN)
    self.assertEqual(car.base_price, 2500000)
    self.assertEqual(car.car_type, CarType.SEDAN)
    self.assertEqual(car.get_price(), 2500000) # Без опций

def test_car_with_options(self):
    """Тест автомобиля с опциями"""
    car = Car("Honda Civic", 2000000, CarType.SEDAN)
    option1 = CarPart("Климат-контроль", 40000)
    option2 = CarPart("Кожанный салон", 80000)

    car.add_option(option1)
    car.add_option(option2)

    self.assertEqual(car.get_price(), 2120000) # 2000000 + 40000 + 80000
    self.assertEqual(len(car.options), 2)

def test_empty_package(self):
    """Тест пустого пакета"""
    package = CarPackage("Пустой пакет")
    self.assertEqual(package.get_price(), 0)
    self.assertEqual(len(package.children), 0)

def test_package_with_cars(self):
    """Тест пакета с автомобилями"""
    package = CarPackage("Тестовый пакет")
    car1 = Car("Car1", 1000000, CarType.SEDAN)
    car2 = Car("Car2", 2000000, CarType.SUV)

    package.add(car1)
    package.add(car2)

    self.assertEqual(package.get_price(), 3000000)
    self.assertEqual(len(package.children), 2)

def test_nested_packages(self):
    """Тест вложенных пакетов"""
    inner_package = CarPackage("Внутренний пакет")
    inner_package.add(Car("Small Car", 1500000, CarType.SEDAN))

    outer_package = CarPackage("Внешний пакет")
    outer_package.add(inner_package)
    outer_package.add(Car("Big Car", 3000000, CarType.SUV))

```

```
        self.assertEqual(outer_package.get_price(), 4500000)

if __name__ == '__main__':
    unittest.main()
```

tests/tests_mock.py

```
import unittest
from unittest.mock import Mock
from composite import CarPackage, Car

class TestAutosalonMocks(unittest.TestCase):
    """Mock тесты для автосалона"""

    def test_package_with_mock_cars(self):
        """Тест пакета с mock автомобилями"""
        package = CarPackage("Тестовый пакет")

        # Создаем mock автомобили
        mock_car1 = Mock(spec=Car)
        mock_car1.get_price.return_value = 1000000

        mock_car2 = Mock(spec=Car)
        mock_car2.get_price.return_value = 2000000

        package.add(mock_car1)
        package.add(mock_car2)

        self.assertEqual(package.get_price(), 3000000)
        mock_car1.get_price.assert_called()
        mock_car2.get_price.assert_called()

    def test_nested_mock_packages(self):
        """Тест вложенных пакетов с mock"""
        mock_inner_package = Mock(spec=CarPackage)
        mock_inner_package.get_price.return_value = 2500000

        outer_package = CarPackage("Внешний пакет")
        outer_package.add(mock_inner_package)

        self.assertEqual(outer_package.get_price(), 2500000)
        mock_inner_package.get_price.assert_called()

if __name__ == '__main__':
    unittest.main()
```

tests/features/steps/tests_BDD.py

```
from behave import given, when, then
import sys
import os
```

```
sys.path.append(os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__)))))
```

```
from composite import CarPart, Car, CarPackage, CarType
```

```
car_parts = {}
cars = {}
packages = {}
current_package = None
```

```
@given("I have a car part \"{name}\" with price {price}")
def step_create_car_part(context, name, price):
    car_parts[name] = CarPart(name, float(price))
```

```
@given("I have a car \"{model}\" with base price {price} and type \"{car_type}\"")
def step_create_car(context, model, price, car_type):
    cars[model] = Car(model, float(price), CarType[car_type.upper()])
```

```
@given("I have a car package \"{name}\"")
def step_create_car_package(context, name):
    packages[name] = CarPackage(name)
    context.current_package = packages[name]
```

```
@when("I add \"{part_name}\" to car")
def step_add_part_to_car(context, part_name):
    current_car = list(cars.values())[-1]
    current_car.add_option(car_parts[part_name])
```

```
@when("I add \"{car_name}\" to package")
def step_add_car_to_package(context, car_name):
    if hasattr(context, 'current_package') and context.current_package:
        current_package = context.current_package
    else:
        current_package = list(packages.values())[-1]
    current_package.add(cars[car_name])
    context.current_package = current_package
```

```
@when("I add \"{car_name}\" to \"{package_name}\" package")
def step_add_car_to_specific_package(context, car_name, package_name):
    packages[package_name].add(cars[car_name])
    context.current_package = packages[package_name]
```

```
@when("I add \"{source_package}\" package to \"{target_package}\" package")
```

```

def step_add_package_to_package(context, source_package, target_package):
    packages[target_package].add(packages[source_package])
    context.current_package = packages[target_package]

@then('the part price should be {expected_price}')
def step_check_part_price(context, expected_price):
    part_name = list(car_parts.keys())[-1]
    actual_price = car_parts[part_name].get_price()
    assert actual_price == float(expected_price), f"Expected {expected_price}, got {actual_price}"

@then('the part description should contain "{text}"')
def step_check_part_description(context, text):
    part_name = list(car_parts.keys())[-1]
    description = car_parts[part_name].get_description()
    assert text in description, f"Text '{text}' not found in: {description}"

@then('the car base price should be {expected_price}')
def step_check_car_base_price(context, expected_price):
    car_model = list(cars.keys())[-1]
    actual_price = cars[car_model].base_price
    assert actual_price == float(expected_price), f"Expected {expected_price}, got {actual_price}"

@then('the car type should be "{expected_type}"')
def step_check_car_type(context, expected_type):
    car_model = list(cars.keys())[-1]
    actual_type = cars[car_model].car_type.value
    assert actual_type == expected_type.lower(), f"Expected {expected_type}, got {actual_type}"

@then('the car total price should be {expected_price}')
def step_check_car_total_price(context, expected_price):
    car_model = list(cars.keys())[-1]
    actual_price = cars[car_model].get_price()
    assert actual_price == float(expected_price), f"Expected {expected_price}, got {actual_price}"

@then('the car should have {count} options')
def step_check_car_options_count(context, count):
    car_model = list(cars.keys())[-1]
    actual_count = len(cars[car_model].options)
    assert actual_count == int(count), f"Expected {count} options, got {actual_count}"

@then('the package should be empty')
def step_check_package_empty(context):
    package = context.current_package if hasattr(context, 'current_package') else
list(packages.values())[-1]
    assert len(package.children) == 0, f"Package is not empty, contains {len(package.children)}
items"

@then('the package price should be {expected_price}')
def step_check_package_price(context, expected_price):

```



```

package = context.current_package if hasattr(context, 'current_package') else
list(packages.values())[-1]
actual_price = package.get_price()
assert actual_price == float(expected_price), f'Expected {expected_price}, got {actual_price}'

@then('the package should contain {count} cars')
def step_check_package_car_count(context, count):
    package = context.current_package if hasattr(context, 'current_package') else
list(packages.values())[-1]
    actual_count = len(package.children)
    assert actual_count == int(count), f'Expected {count} cars, got {actual_count}'

@then('{package_name}' package total price should be {expected_price}')
def step_check_specific_package_price(context, package_name, expected_price):
    actual_price = packages[package_name].get_price()
    assert actual_price == float(expected_price), f'Expected {expected_price}, got {actual_price}'

@then('{package_name}' package should contain {count} sub-packages')
def step_check_package_subpackages_count(context, package_name, count):
    package = packages[package_name]
    # Считаем только дочерние пакеты (не автомобили)
    subpackages_count = sum(1 for child in package.children if isinstance(child, CarPackage))
    assert subpackages_count == int(count), f'Expected {count} sub-packages, got
{subpackages_count}'

```

tests/features/tests_BDD.feature

Feature: Composite Pattern for Autosalon Management

As an autosalon manager

I want to use composite pattern to manage cars and packages

So that I can handle both individual cars and complex car packages

Scenario: Create a car part

Given I have a car part "Колесо" with price 15000

Then the part price should be 15000

And the part description should contain "Колесо"

Scenario: Create a basic car

Given I have a car "Toyota Corolla" with base price 1800000 and type "sedan"

Then the car base price should be 1800000

And the car type should be "sedan"

Scenario: Add options to car

Given I have a car "Honda Civic" with base price 2000000 and type "sedan"

And I have a car part "Климат-контроль" with price 40000

And I have a car part "Кожаный руль" with price 15000

When I add "Климат-контроль" to car

And I add "Кожаный руль" to car

Then the car total price should be 2055000

And the car should have 2 options

Scenario: Create empty car package

Given I have a car package "Базовый пакет"

Then the package should be empty

And the package price should be 0

Scenario: Add cars to package

Given I have a car package "Семейные автомобили"

And I have a car "Toyota Camry" with base price 2500000 and type "sedan"

And I have a car "Honda CR-V" with base price 3200000 and type "suv"

When I add "Toyota Camry" to package

And I add "Honda CR-V" to package

Then the package should contain 2 cars

Scenario: Create nested packages

Given I have a car package "Эконом класс"

And I have a car "Kia Rio" with base price 1500000 and type "sedan"

And I have a car package "Премиум класс"

And I have a car "BMW X5" with base price 6000000 and type "suv"

And I have a car package "Автосалон"

When I add "Kia Rio" to "Эконом класс" package

And I add "BMW X5" to "Премиум класс" package

And I add "Эконом класс" package to "Автосалон" package

And I add "Премиум класс" package to "Автосалон" package

Then "Автосалон" package total price should be 7500000

And "Автосалон" package should contain 2 sub-packages

main.py

```
from composite import create_sample_autosalon

def main():
    print("=== АВТОСАЛОН - ДЕМОНСТРАЦИЯ COMPOSITE PATTERN ===\n")

    autosalon = create_sample_autosalon()

    print("СТРУКТУРА АВТОСАЛОНА:")
    print("=" * 50)
    print(autosalon.get_description())

if __name__ == "__main__":
    main()
```

Скриншоты работы:

```
(base) an.akopov@MacBook-Air-Akopov lab6 % python3 main.py
=== АВТОСАЛОН – ДЕМОНСТРАЦИЯ COMPOSITE PATTERN ===

СТРУКТУРА АВТОСАЛОНА:
=====
Пакет: Автосалон 'Престиж'
  Пакет: Семейный пакет
    Автомобиль: Toyota Camry (sedan) – 2500000 руб.
    Доп. опции:
      – Запчасть: Кожаные сиденья – 50000 руб.
      – Запчасть: Навигационная система – 30000 руб.
    Автомобиль: Honda CR-V (suv) – 3200000 руб.
    Доп. опции:
      – Запчасть: Навигационная система – 30000 руб.
  Общая стоимость пакета: 5810000 руб.
  Пакет: Премиум пакет
    Автомобиль: Porsche 911 (sport) – 8500000 руб.
    Доп. опции:
      – Запчасть: Кожаные сиденья – 50000 руб.
      – Запчасть: Спортивный обвес – 70000 руб.
  Общая стоимость пакета: 8620000 руб.
Общая стоимость пакета: 14430000 руб.
(base) an.akopov@MacBook-Air-Akopov lab6 %
```

Тесты:

```
(base) an.akopov@MacBook-Air-Akopov lab6 % python3 -m unittest tests.tests_mocks
..
-----
Ran 2 tests in 0.001s

OK
(base) an.akopov@MacBook-Air-Akopov lab6 %
```

```

● (base) an.akopov@MacBook-Air-Akopov lab6 % behave tests/features/
USING RUNNER: behave.runner:Runner
Feature: Composite Pattern for Autosalon Management # tests/features/tests_BDD.feature:1
  As an autosalon manager
  I want to use composite pattern to manage cars and packages
  So that I can handle both individual cars and complex car packages
  Scenario: Create a car part # tests/features/tests_BDD.feature:6
    Given I have a car part "Колесо" with price 15000 # tests/features/steps/tests_BDD.py:16 0.000s
    Then the part price should be 15000 # tests/features/steps/tests_BDD.py:53 0.000s
    And the part description should contain "Колесо" # tests/features/steps/tests_BDD.py:59 0.000s

  Scenario: Create a basic car # tests/features/tests_BDD.feature:11
    Given I have a car "Toyota Corolla" with base price 1800000 and type "sedan" # tests/features/steps/tests_BDD.py:20 0.000s
    Then the car base price should be 1800000 # tests/features/steps/tests_BDD.py:65 0.000s
    And the car type should be "sedan" # tests/features/steps/tests_BDD.py:71 0.000s

  Scenario: Add options to car # tests/features/tests_BDD.feature:16
    Given I have a car "Honda Civic" with base price 2000000 and type "sedan" # tests/features/steps/tests_BDD.py:20 0.000s
    And I have a car part "Климат-контроль" with price 40000 # tests/features/steps/tests_BDD.py:16 0.000s
    And I have a car part "Кожаный руль" with price 15000 # tests/features/steps/tests_BDD.py:16 0.000s
    When I add "Климат-контроль" to car # tests/features/steps/tests_BDD.py:29 0.000s
    And I add "Кожаный руль" to car # tests/features/steps/tests_BDD.py:29 0.000s
    Then the car total price should be 2055000 # tests/features/steps/tests_BDD.py:77 0.000s
    And the car should have 2 options # tests/features/steps/tests_BDD.py:83 0.000s

  Scenario: Create empty car package # tests/features/tests_BDD.feature:25
    Given I have a car package "Базовый пакет" # tests/features/steps/tests_BDD.py:24 0.000s
    Then the package should be empty # tests/features/steps/tests_BDD.py:89 0.000s
    And the package price should be 0 # tests/features/steps/tests_BDD.py:94 0.000s

  Scenario: Add cars to package # tests/features/tests_BDD.feature:30
    Given I have a car package "Семейные автомобили" # tests/features/steps/tests_BDD.py:24 0.000s
    And I have a car "Toyota Camry" with base price 2500000 and type "sedan" # tests/features/steps/tests_BDD.py:20 0.000s
    And I have a car "Honda CR-V" with base price 3200000 and type "suv" # tests/features/steps/tests_BDD.py:20 0.000s
    When I add "Toyota Camry" to package # tests/features/steps/tests_BDD.py:34 0.000s
    And I add "Honda CR-V" to package # tests/features/steps/tests_BDD.py:34 0.000s
    Then the package should contain 2 cars # tests/features/steps/tests_BDD.py:100 0.000s

  Scenario: Create nested packages # tests/features/tests_BDD.feature:39
    Given I have a car package "Эконом класс" # tests/features/steps/tests_BDD.py:24 0.000s
    And I have a car "Kia Rio" with base price 1500000 and type "sedan" # tests/features/steps/tests_BDD.py:20 0.000s
    And I have a car package "Премиум класс" # tests/features/steps/tests_BDD.py:24 0.000s
    And I have a car "BMW X5" with base price 6000000 and type "suv" # tests/features/steps/tests_BDD.py:20 0.000s
    And I have a car package "Автосалон" # tests/features/steps/tests_BDD.py:24 0.000s
    When I add "Kia Rio" to "Эконом класс" package # tests/features/steps/tests_BDD.py:43 0.000s
    And I add "BMW X5" to "Премиум класс" package # tests/features/steps/tests_BDD.py:43 0.000s
    And I add "Эконом класс" package to "Автосалон" package # tests/features/steps/tests_BDD.py:48 0.000s
    And I add "Премиум класс" package to "Автосалон" package # tests/features/steps/tests_BDD.py:48 0.000s
    Then "Автосалон" package total price should be 7500000 # tests/features/steps/tests_BDD.py:106 0.000s
    And "Автосалон" package should contain 2 sub-packages # tests/features/steps/tests_BDD.py:111 0.000s

1 feature passed, 0 failed, 0 skipped
6 scenarios passed, 0 failed, 0 skipped
33 steps passed, 0 failed, 0 skipped

```

```

● (base) an.akopov@MacBook-Air-Akopov lab6 % python3 -m unittest tests.tests_TDD
.....

Ran 6 tests in 0.000s

OK
● (base) an.akopov@MacBook-Air-Akopov lab6 %

```